TECHNICAL ARCHITECTURE SPECIFICATION

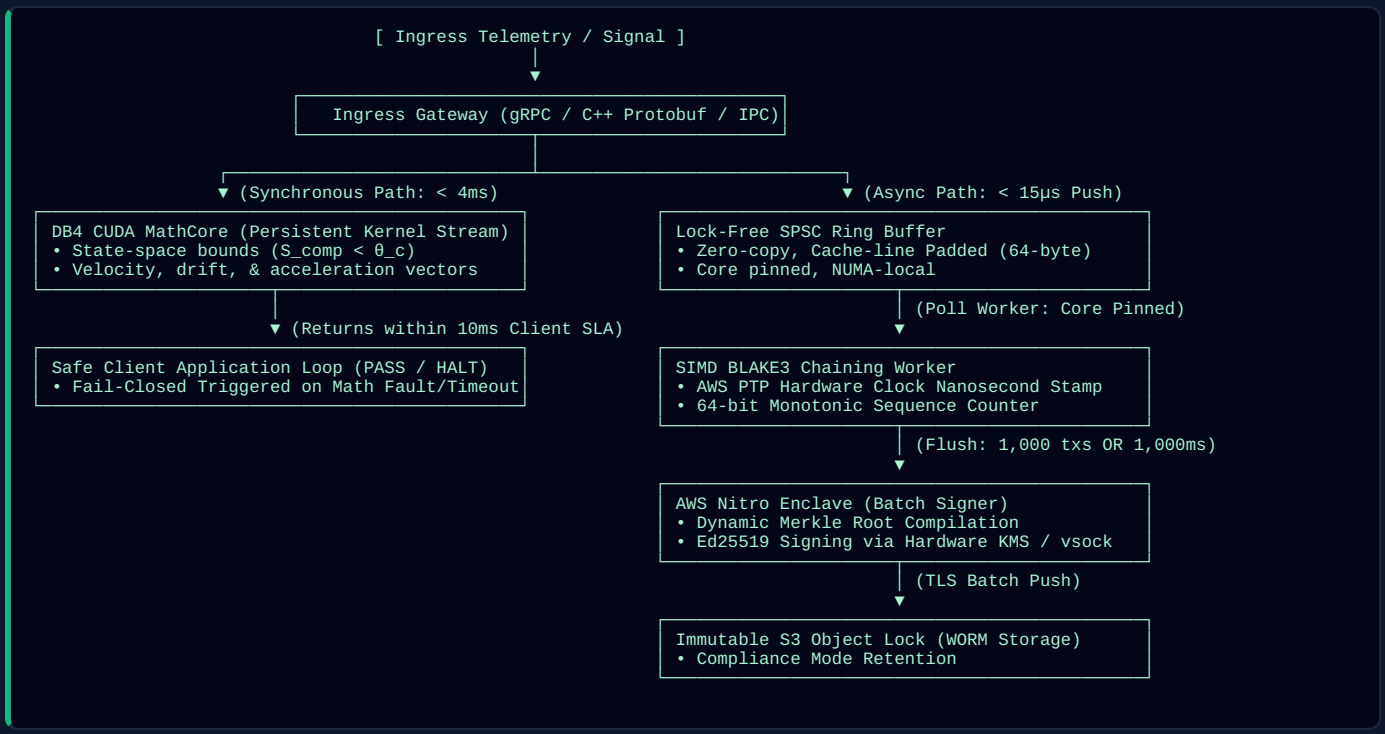
DB4 MathCore Architecture Specification (v2.0)

Low-Latency Deterministic AI Safety Middleware

DOCUMENT ID	ENTITY	TARGET AUDIENCE	SLA GUARANTEE
DB4-SPEC-2026-V2	2639389 Alberta Ltd.	AWS Solutions Architects, Systems Engineers	< 4.0ms Deterministic Response

1. EXECUTIVE SUMMARY & CORE GUARANTEES

DB4 MathCore is a low-latency C++/CUDA deterministic safety middleware designed for dynamic AI agents, industrial control loops, and critical simulation systems. It solves the fundamental trade-off between sub-millisecond safety state-space evaluations ($S_{composite} < \theta_{critical}$) and non-repudiable, tamper-evident cryptographic audit trailing.



Synchronous Path Budget

Complete evaluation and response returned within **< 4 ms** (inside the client application's 10 ms SLA envelope).

Ingress Latch Overhead

Non-blocking ring-buffer dispatch executed in **< 15 µs** using zero-copy memory primitives.

Audit Trail Non-Repudiation

Cryptographically anchored via SIMD BLAKE3, AWS PTP hardware time, and isolated Nitro Enclave Ed25519 signatures committed to S3 Object Lock.

2. INGRESS & SYNCHRONOUS EVALUATION PATH

Transport Layer Specifications: Replaces bare JSON-over-HTTP endpoints (`POST /api/v1/signal`) with gRPC over HTTP/2 with binary Protocol Buffers or direct local shared memory IPC (`POSIX shm_open`) for co-located containers.

Ingress Serialization Budget: Serialization/deserialization latency bounded to **≤ 120 µs** using static buffer allocations (avoiding dynamic heap allocation in the request path).

CUDA Execution Pipeline Memory Management: Memory mappings utilize `cudaHostAllocMapped` with `cudaHostAllocWriteCombined` to guarantee zero-copy host-to-device memory access across the PCIe bus.

Kernel Dispatch Strategy: To eliminate CUDA runtime launch overhead (typically 5–15 µs per kernel), the core runs a persistent execution kernel loop synchronized via atomic control flags (`cudaStreamWaitEvent`), keeping Tensor Cores hot and deterministic.

Mathematical Boundary Check:

$$S_{composite} = \alpha \, \|v(t)\| + \beta \, \|a(t)\| + \gamma \, D_{drift}(x_t, x_{target})$$

The system computes whether $S_{composite} < \theta_{critical}$. If the condition is met, an immediate `PASS` signal is returned. If $S_{composite} \geq \theta_{critical}$ or if any CUDA hardware fault/NaN occurs, the path drops immediately to a fail-closed `HALT`.

3. ASYNCHRONOUS CRYPTOGRAPHIC AUDIT ARCHITECTURE

Lock-Free SPSC Ring Buffer Topology: Fixed-size single-producer single-consumer (SPSC) circular ring buffer mapped into pinned host memory.

Cache Alignment: Internal read and write cursors are padded to distinct 64-byte boundaries (cache lines) to eliminate false sharing between the ingress thread and the background hashing worker.

Backpressure & Saturation Policy (Fail-Closed):

METRIC / TRIGGER	THRESHOLD VALUE	SYSTEM OPERATIONAL ACTION
Ring Buffer Capacity	`262,144 slots` (2^{18} entries)	~64 MB total pinned memory footprint allocated on boot.
High-Water Mark	`90% capacity`	Emits a degraded latency alert to supervisory metrics daemon.
Ring Saturation	`100% capacity`	Triggers fail-closed circuit breaker; returns `HALT` to prevent un-audited telemetry.

Hardware-Anchored Time & SIMD Hashing:

Clock Source: AWS Time Sync Service via `/dev/ptp0` (PTP hardware clock) providing sub-microsecond reference timestamps cross-verified with an internal 64-bit monotonic sequence counter.

Hash Chaining: A dedicated CPU-pinned worker executes **AVX2 SIMD-accelerated BLAKE3** (optimized for 2nd Gen AMD EPYC Rome on `g5` instances; optional AVX-512 target on Intel `g4dn` / Zen 4 nodes). Each entry binds:

$$H_i = BLAKE3(\, H_{i-1} \,\|\, Timestamp_{PTP} \,\|\, SeqCounter \,\|\, Payload_{raw} \,)$$

4. NITRO ENCLAVE & IMMUTABLE STORAGE FLOW

ATTRIBUTE	SPECIFICATION	OPERATIONAL TARGET
Enclave Isolation	AWS Nitro Enclave (No external IP, vsock channel only)	Hardware-enforced isolation from host OS kernel
Signing Key Security	Ed25519 private key decrypted into Enclave memory via AWS KMS via Attestation PCRs	Zero developer or host access; key never leaves Enclave memory boundary
Batch Aggregation	Merkle Tree construction computed over batch windows	Every 1,000 transactions OR 1,000 ms interval
Commit Payload	Merkle Root + Range Timestamps + Ed25519 Signature	~4 KB canonical batch manifest
Storage Target	S3 Object Lock in Compliance Mode (WORM Storage)	Immutable legal hold; retention policy: 7 years

5. INFRASTRUCTURE SIZING & HARDWARE TOPOLOGY

```
Target AWS Instance: g5.4xlarge (16 vCPUs / 8 Physical Cores + NVIDIA A10G 24GB)

Host OS vCPU Thread Mapping (Core Complex / SMT Aware via /sys/devices/system/cpu/cpu*/topology/thread_siblings_list) ├─ vCPUs 0-3 (Physical Cores 0-1): OS Services, System Interrupts, Metrics Exporters ├─ vCPUs 4-7 (Physical Cores 2-3): Ingress Signal Gate (gRPC Event Loop, epoll) ├─ vCPUs 8-9 (Physical Core 4): CUDA Memory Dispatcher (Host-to-Device Streaming) └─ vCPUs 10-11 (Physical Core 5): AVX2 BLAKE3 Chaining Worker (PTP Clock Consumer)
Nitro Enclave vCPU Allocation (Explicit Enclave Carve-Out) └─ vCPUs 12-15 (Physical Cores 6-7): Isolated vCPUs for Merkle Compaction & Signing
Accelerator & Cache Locality Governance ├─ NVIDIA A10G: Locked Base/Boost Clocks (nvidia-smi -lgc), Pre-warmed TensorRT Contexts └─ CPU Thread Affinity: Strict pthread_setaffinity_np binding within shared CCX L3 boundaries
```

Thread Affinity & SMT Governance: Ingress and ring-buffer reader threads use explicit `pthread_setaffinity_np` calls bound to dedicated logical vCPU pairs to maintain cache locality and prevent context migration across AMD Core Complex dies (CCXs).

TECHNICAL ARCHITECTURE QUERIES FOR AWS SOLUTIONS ARCHITECT

1. Does AWS have verified throughput/latency benchmarks for vsock proxies bridging the Nitro Enclave to AWS KMS and S3 endpoints under high-frequency batch flushes?
2. Given our jitter-sensitive polling loops, would you recommend stepping up to `g5.8xlarge` (32 vCPUs / 16 cores) to ensure Nitro Enclave vCPU carving does not share SMT hyperthread siblings with our ingress or CUDA dispatch loops?
3. Are there recommended patterns for running AWS Time Sync (`/dev/ptp0`) inside or alongside Nitro Enclaves without introducing time virtualization drift?